



Parser combinator

In computer programming, a **parser combinator** is a higher-order function that accepts several parsers as input and returns a new parser as its output. In this context, a parser is a function accepting strings as input and returning some structure as output, typically a parse tree or a set of indices representing locations in the string where parsing stopped successfully. Parser combinators enable a recursive descent parsing strategy that facilitates modular piecewise construction and testing. This parsing technique is called **combinatory parsing**.

Parsers using combinators have been used extensively in the prototyping of compilers and processors for domain-specific languages such as natural-language user interfaces to databases, where complex and varied semantic actions are closely integrated with syntactic processing. In 1989, Richard Frost and John Launchbury demonstrated^[1] use of parser combinators to construct natural-language interpreters. Graham Hutton also used higher-order functions for basic parsing in 1992^[2] and monadic parsing in 1996.^[3] S. D. Swierstra also exhibited the practical aspects of parser combinators in 2001.^[4] In 2008, Frost, Hafiz and Callaghan^[5] described a set of parser combinators in the functional programming language Haskell that solve the long-standing problem of accommodating left recursion, and work as a complete top-down parsing tool in polynomial time and space.

Basic idea

In any programming language that has first-class functions, parser combinators can be used to combine basic parsers to construct parsers for more complex rules. For example, a production rule of a context-free grammar (CFG) may have one or more alternatives and each alternative may consist of a sequence of non-terminal(s) and/or terminal(s), or the alternative may consist of a single non-terminal or terminal or the empty string. If a simple parser is available for each of these alternatives, a parser combinator can be used to combine each of these parsers, returning a new parser which can recognise any or all of the alternatives.

In languages that support operator overloading, a parser combinator can take the form of an infix operator, used to glue different parsers to form a complete rule. Parser combinators thereby enable parsers to be defined in an embedded style, in code which is similar in structure to the rules of the formal grammar. As such, implementations can be thought of as executable specifications with all the associated advantages such as readability.

The combinators

To keep the discussion relatively straightforward, we discuss parser combinators in terms of *recognizers* only. If the input string is of length `#input` and its members are accessed through an index `j`, a recognizer is a parser which returns, as output, a set of indices representing

indices at which the parser successfully finished recognizing a sequence of tokens that begin at index j . An empty result set indicates that the recognizer failed to recognize any sequence beginning at index j .

- The empty recognizer recognizes the empty string. This parser always succeeds, returning a singleton set containing the input index:

$$\text{empty}(j) = \{j\}$$

- A recognizer term x recognizes the terminal x . If the token at index j in the input string is x , this parser returns a singleton set containing $j + 1$; otherwise, it returns the empty set.

$$\text{term}(x, j) = \begin{cases} \{\}, & j \geq \#input \\ \{j + 1\}, & j^{\text{th}} \text{ element of } input = x \\ \{\}, & \text{otherwise} \end{cases}$$

Given two recognizers p and q , we can define two major parser combinators, one for matching alternative rules and one for sequencing rules:

- The 'alternative' parser combinator, $\oplus$, applies each of the recognizers on the same index j and returns the union of the finishing indices of the recognizers:

$$(p \oplus q)(j) = p(j) \cup q(j)$$

- The 'sequence' combinator, $\otimes$, applies the first recognizer p to the input index j , and for each finishing index applies the second recognizer q with that as a starting index. It returns the union of the finishing indices returned from all invocations of q :

$$(p \otimes q)(j) = \bigcup \{q(k) : k \in p(j)\}$$

There may be multiple distinct ways to parse a string while finishing at the same index, indicating an ambiguous grammar. Simple recognizers do not acknowledge these ambiguities; each possible finishing index is listed only once in the result set. For a more complete set of results, a more complicated object such as a parse tree must be returned.

Examples

Consider a highly ambiguous context-free grammar, $s ::= 'x' s s \mid \epsilon$. Using the combinators defined earlier, we can modularly define executable notations of this grammar in a modern functional programming language (e.g., Haskell) as $s = \text{term } 'x' <*> s <*> s <+> \text{empty}$. When the recognizer s is applied at index 2 of the input sequence $x \ x \ x \ x \ x$ it would return a result set $\{2, 3, 4, 5\}$, indicating that there were matches starting at index 2 and finishing at any index between 2 and 5 inclusive.

Shortcomings and solutions

Parser combinators, like all recursive descent parsers, are not limited to the context-free grammars and thus do no global search for ambiguities in the LL(k) parsing First_k and Follow_k sets. Thus, ambiguities are not known until run-time if and until the input triggers them. In such cases, the recursive descent parser may default (perhaps unknown to the grammar designer) to one of the possible ambiguous paths, resulting in semantic confusion (aliasing) in the use of the language. This leads to bugs by users of ambiguous programming languages, which are not reported at compile-time, and which are introduced not by human error, but by ambiguous grammar. The only solution that eliminates these bugs is to remove the ambiguities and use a context-free grammar.

The simple implementations of parser combinators have some shortcomings, which are common in top-down parsing. Naïve combinatory parsing requires exponential time and space when parsing an ambiguous context-free grammar. In 1996, Frost and Szydlowski demonstrated how memoization can be used with parser combinators to reduce the time complexity to polynomial.^[6] Later Frost used monads to construct the combinators for systematic and correct threading of memo-table throughout the computation.^[7]

Like any top-down recursive descent parsing, the conventional parser combinators (like the combinators described above) will not terminate while processing a left-recursive grammar (e.g. $s ::= s \text{ <*> term } 'x' | \text{empty}$). A recognition algorithm that accommodates ambiguous grammars with direct left-recursive rules is described by Frost and Hafiz in 2006.^[8] The algorithm curtails the otherwise ever-growing left-recursive parse by imposing depth restrictions. That algorithm was extended to a complete parsing algorithm to accommodate indirect as well as direct left-recursion in polynomial time, and to generate compact polynomial-size representations of the potentially exponential number of parse trees for highly ambiguous grammars by Frost, Hafiz and Callaghan in 2007.^[9] This extended algorithm accommodates indirect left recursion by comparing its ‘computed context’ with ‘current context’. The same authors also described their implementation of a set of parser combinators written in the Haskell language based on the same algorithm.^{[5][10]}

Notes

1. Frost & Launchbury 1989.
2. Hutton 1992.
3. Hutton, Graham; Meijer, Erik. Monadic Parser Combinators (<http://www.cs.nott.ac.uk/~pszg/mh/monparsing.pdf>) (PDF) (Report). University of Nottingham. Retrieved 13 February 2023.
4. Swierstra 2001.
5. Frost, Hafiz & Callaghan 2008.
6. Frost & Szydlowski 1996.
7. Frost 2003.
8. Frost & Hafiz 2006.
9. Frost, Hafiz & Callaghan 2007.
10. cf. X-SAIGA (<http://www.cs.uwindsor.ca/~hafiz/proHome.html>) — **executable specifications of grammars**

References

- Burge, William H. (1975). *Recursive Programming Techniques* (<https://archive.org/details/recursiveprogram0000burg>). The Systems programming series. Addison-Wesley. ISBN 978-0201144505.
- Frost, Richard; Launchbury, John (1989). "Constructing natural language interpreters in a lazy functional language" (<https://web.archive.org/web/20130606162214/https://courses.cit.ornell.edu/ling4424/frost-launchbury.pdf>) (PDF). *The Computer Journal*. Special edition on Lazy Functional Programming. **32** (2): 108–121. doi:10.1093/comjnl/32.2.108 (<https://doi.org/10.1093%2Fcomjnl%2F32.2.108>). Archived from the original on 2013-06-06.
- Frost, Richard A.; Szydlowski, Barbara (1996). "Memoizing Purely Functional Top-Down Backtracking Language Processors" (http://cs.uwindsor.ca/~richard/PUBLICATIONS/SCOMP_96.pdf) (PDF). *Sci. Comput. Program.* **27** (3): 263–288. doi:10.1016/0167-6423(96)00014-7 (<https://doi.org/10.1016%2F0167-6423%2896%2900014-7>).
- Frost, Richard A. (2003). "Monadic Memoization towards Correctness-Preserving Reduction of Search". *Proceedings of the 16th Canadian Society for Computational Studies of Intelligence Conference on Advances in Artificial Intelligence (AI'03)* (http://cs.uwindsor.ca/~richard/PUBLICATIONS/AI_03.pdf) (PDF). Springer. pp. 66–80. ISBN 978-3-540-40300-5.
- Frost, Richard A.; Hafiz, Rahmatullah (2006). "A New Top-Down Parsing Algorithm to Accommodate Ambiguity and Left Recursion in Polynomial Time" (<http://cs.uwindsor.ca/~hafiz/pub/p46-frost.pdf>) (PDF). *ACM SIGPLAN Notices*. **41** (5): 46–54. doi:10.1145/1149982.1149988 (<https://doi.org/10.1145%2F1149982.1149988>). S2CID 8006549 (<https://api.semanticscholar.org/CorpusID:8006549>).
- Frost, Richard A.; Hafiz, Rahmatullah; Callaghan, Paul (2007). "Modular and Efficient Top-Down Parsing for Ambiguous Left-Recursive Grammars". *Proceedings of the 10th International Workshop on Parsing Technologies (IWPT), ACL-SIGPARSE*: 109–120. CiteSeerX 10.1.1.97.8915 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.97.8915>).
- Frost, Richard A.; Hafiz, Rahmatullah; Callaghan, Paul (2008). "Parser Combinators for Ambiguous Left-Recursive Grammars". *Practical Aspects of Declarative Languages*. ACM-SIGPLAN. Vol. 4902. pp. 167–181. CiteSeerX 10.1.1.89.2132 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.89.2132>). doi:10.1007/978-3-540-77442-6_12 (https://doi.org/10.1007%2F978-3-540-77442-6_12). ISBN 978-3-540-77441-9.
- Hutton, Graham (1992). "Higher-order functions for parsing". *Journal of Functional Programming*. **2** (3): 323–343. CiteSeerX 10.1.1.34.1287 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.34.1287>). doi:10.1017/s0956796800000411 (<https://doi.org/10.1017%2Fs0956796800000411>). S2CID 31067887 (<https://api.semanticscholar.org/CorpusID:31067887>).
- Okasaki, Chris (1998). "Even higher-order functions for parsing or Why would anyone ever want to use a sixth-order function?" (<https://doi.org/10.1017%2FS0956796898003001>). *Journal of Functional Programming*. **8** (2): 195–199. doi:10.1017/S0956796898003001 (<https://doi.org/10.1017%2FS0956796898003001>). S2CID 59694674 (<https://api.semanticscholar.org/CorpusID:59694674>).
- Swierstra, S. Doaitse (2001). "Combinator parsers: From toys to tools" (<https://doi.org/10.1016%2FS1571-0661%2805%2980545-6>). *Electronic Notes in Theoretical Computer Science*. **41**: 38–59. doi:10.1016/S1571-0661(05)80545-6 (<https://doi.org/10.1016%2FS1571-0661%2805%2980545-6>).
- Wadler, Philip (1985). "How to replace failure by a list of successes a method for exception handling, backtracking, and pattern matching in lazy functional languages". *Functional Programming Languages and Computer Architecture*. Lecture Notes in Computer Science.

Vol. 201. pp. 113–128. doi:10.1007/3-540-15975-4_33 (https://doi.org/10.1007%2F3-540-15975-4_33). ISBN 978-0-387-15975-1 – via Proceedings of a Conference on Functional Programming Languages and Computer Architecture.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Parser_combinator&oldid=1268750964"